

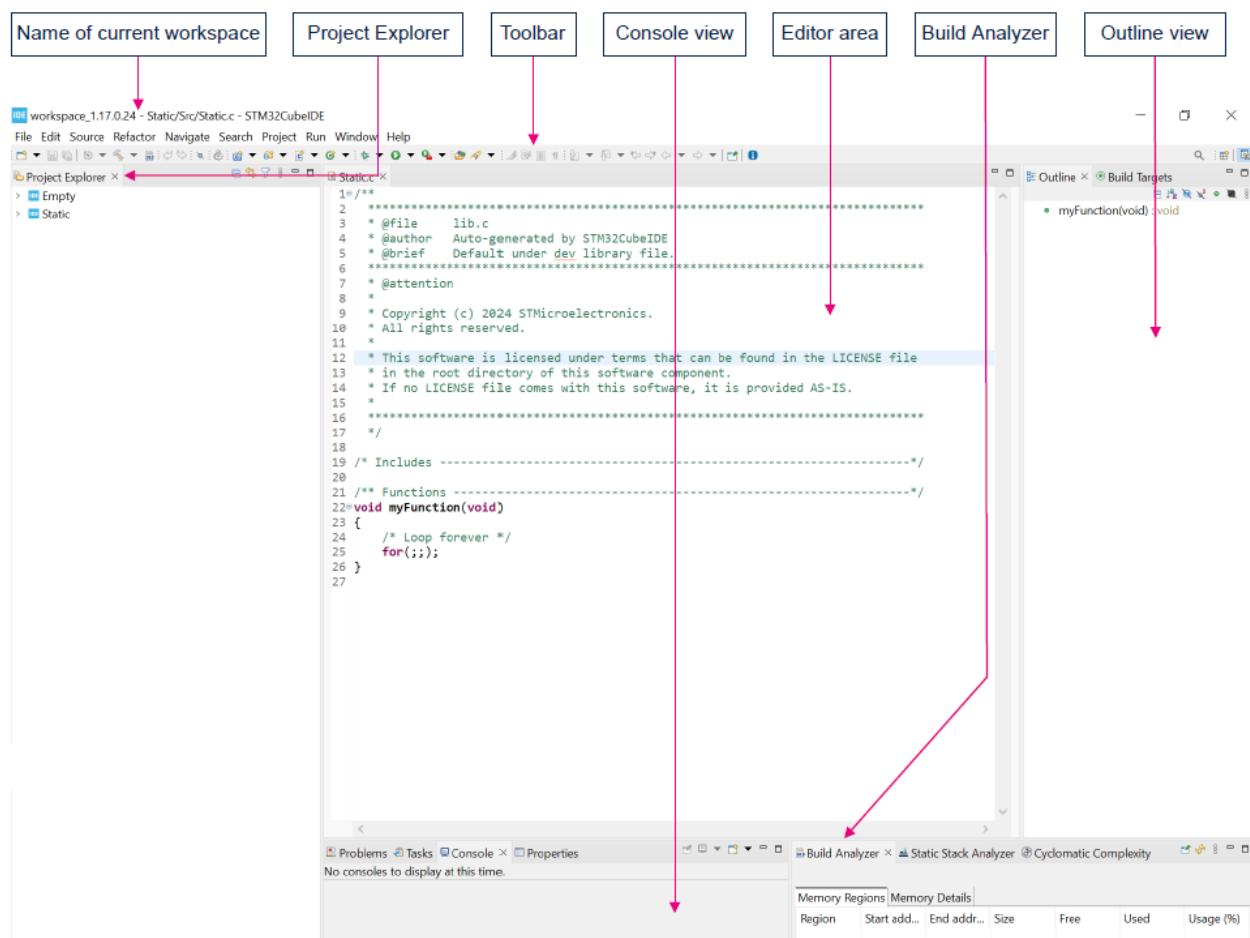
Learning goals:

- Familiarize with the STM32CubeIDE for development and debugging
- Breakpoints and step execution
- Explore the device memory and the processor registers
- First look at the ARM assembly language

STM32IDE hints:

- Registers (Right-click number format: hex/bin)
- Window->Show View-> Disassembly

In this tutorial we will have a first introduction to the STM32CubeIDE environment, we will familiarize ourselves with both the device memory and the processor registers and will have a look at the ARM assembly language.

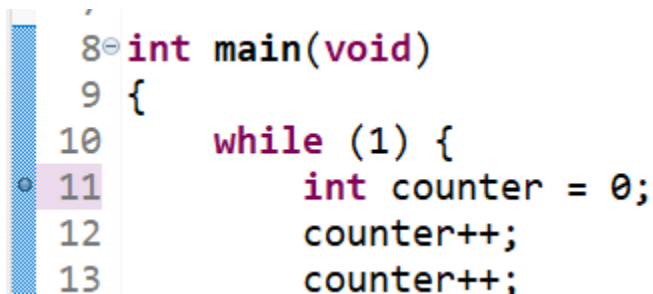


The default STM32CubeIDE window

Copy the following code inside main():

```
int main()
{
    int counter = 0;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
}
```

Now place a breakpoint in front of the counter declaration (to place a breakpoint, double click the line, just to the left of the line number until a blue bullet appears):

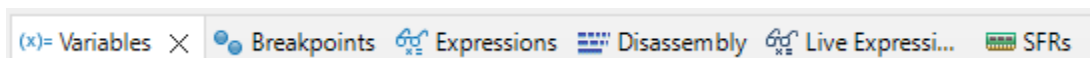


```
8 int main(void)
9 {
10     while (1) {
11         int counter = 0;
12         counter++;
13         counter++;
```

Finally, start a debug session that will allow us to inspect the state of our program:



After compiling, the executable should be loaded to your device and the execution should have started and paused at the highlighted code line where you placed the breakpoint. You might have noticed that the *C/C++ view* has changed to the *Outline view*:



The following tabs are very important, and we will use them a lot: *Variables*, *Breakpoints*, *Disassembly*, *SFRs*. Please take your time exploring the different tabs - what do they do?

Now open the Disassembly view - you should see something similar to the following image:

11		<code>int counter = 0;</code>
0800027e:	<code>movs</code>	<code>r3, #0</code>
08000280:	<code>str</code>	<code>r3, [r7, #4]</code>
12		<code>counter++;</code>
08000282:	<code>ldr</code>	<code>r3, [r7, #4]</code>
08000284:	<code>adds</code>	<code>r3, #1</code>
08000286:	<code>str</code>	<code>r3, [r7, #4]</code>
13		<code>counter++;</code>
08000288:	<code>ldr</code>	<code>r3, [r7, #4]</code>
0800028a:	<code>adds</code>	<code>r3, #1</code>
0800028c:	<code>str</code>	<code>r3, [r7, #4]</code>

The numbers left of the assembly commands (in green) are the memory addresses where the command is stored. The commands in blue are assembly instructions. For example

`movs r3, #0`

means “copy the value 0 to register r3”, while

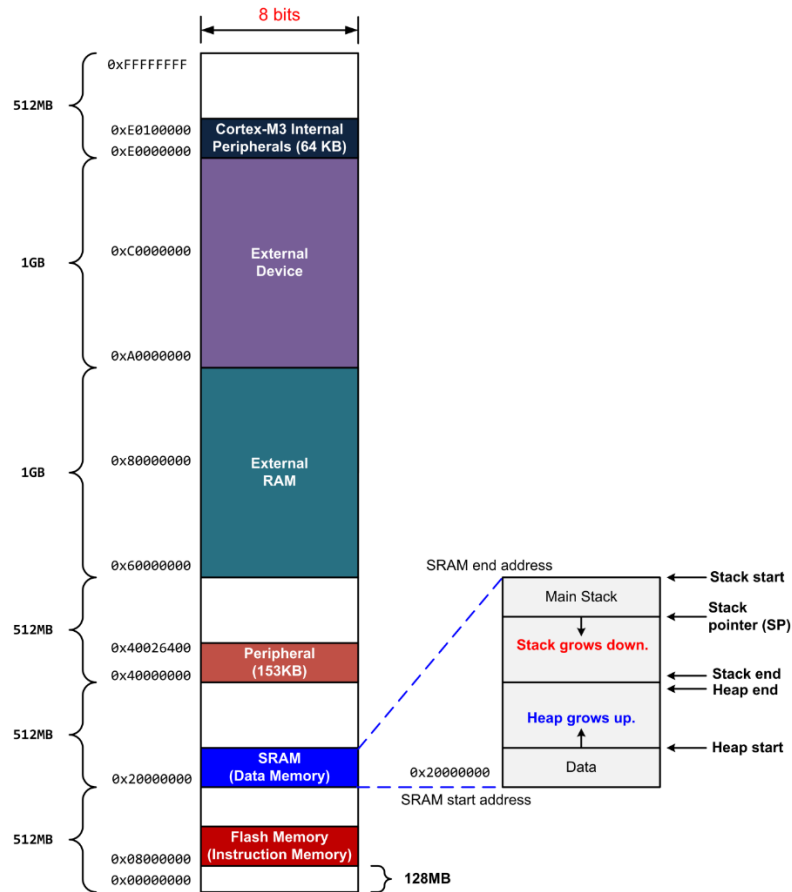
`str r3, [r7, #4]`

means “store the value of register r3 to the memory address that is stored as a value in the r7 register, increased by 4”. We will see more details on the structure of the ARM assembly language starting in module 3.

Now please have a look at the *registers* tab in the left of the screen:

You can examine the values of registers r0-r15 and the xpsr registers. Try to change their number format between decimal, hexadecimal and binary.

Addresses are simply numbers assigned to bytes in memory. To make sure that instructions are **just numbers in memory**, let’s first have a look at the memory.



The memory map of Cortex-M33

Memory is essentially just a table of numbers, with positions numbered sequentially starting from zero (0x00). The sequential numbers (or addresses) are shown in the Memory view. Enter the first memory address (in my case this is 0x0800027E) into the Memory view -> Add Memory Monitor. Right-click to the addresses->Format->Column size->1. Notice the value stored in memory location 0x0800027E:

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
08000270	04	00	00	20	FC	04	00	08	80	B4	8B	B0	00	AF	00	23

Question 1.1: In which part of the memory map is this instruction saved?

Notice that the command is similar to:

MOVS r3, #0

In "Armv8-M Architecture Reference Manual" (do you remember that the B-U585I-IOT02A board has an ARM Cortex-M33 core which follows the Armv8-M architecture?), section **C2.2.1.3 Add, subtract, compare, move (one low register and immediate)** on p483 for the T32/Thumb instruction set we can see the following:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	op			reg			imm8						

This table shows the decode field values and the associated instructions:

op	Instruction
00	MOV (immediate) T1
01	CMP (immediate) T1
10	ADD (immediate) T2
11	SUB (immediate) T2

MOV, CMP, ADD, SUB instruction format

(For now, we will treat MOV and MOVS as the same instruction). If we substitute **op** with 00, **reg** with 011 (since it is r3) and **imm8** with 0, the decoded instruction should have the following 16-bit value:

0b0010 0011 0000 0000

Question 1.2: What is the value in hex?

Similar for the ADDS instruction in the next line of code:

ADDS r3, #1

The decoded instruction should correspond to the following decoded 16-bit value in binary and hex:

0b0011 0011 0000 0001 | 0x3301

In the disassembly view the address of this first ADDS instruction is similar to *0x08000284*. Open the Memory view and check that address location:

Address	0	1	2	3	4	5
08000280	7B	60	7B	68	01	33

Question 1.3: Notice the extra instructions close to MOVS and ADDS. What is their purpose?

11	int counter = 0;
0800027e:	movs r3, #0
08000280:	str r3, [r7, #4]
12	counter++;
08000282:	ldr r3, [r7, #4]
08000284:	adds r3, #1
08000286:	str r3, [r7, #4]
13	counter++;
08000288:	ldr r3, [r7, #4]
0800028a:	adds r3, #1
0800028c:	str r3, [r7, #4]

ARM follows a Load-Store architecture (more for this during the next lectures!). For now, you should know that the *value* of the counter variable is stored in memory. To check this, have a look at the value stored in the r7 register. Open the Registers view:

r7	537657296
r8	-1
r9	-1

Name : r7
Hex: 0x200bffd0
Decimal: 537657296
Octal: 0400277720
Binary: 1000000001011111111111101000
Default: 537657296

Now check the value stored in memory address $0x200bffd0 + 4$:

Address	0	1	2	3	4	5	6	7
200BFFD0	0C	00	00	34	03	00	00	00
200BFFE0	14	05	00	08	00	00	00	00
200BFFF0	00	00	00	00	00	00	00	00

Question 1.4: In which part of the memory is the value of the counter variable stored?

We can instruct the compiler to store the value of a variable directly to a CPU register. Change the definition of counter to *register int counter = 0*; and run the example again.

Question 1.5: What do you notice?

To fix this, trick the compiler to believe that the counter variable is **really useful**! Add the following code after the last counter++; line: *int use_counter = counter*;

Now check again the Disassembly view. What do you notice?

0800027e:	movs	r4, #0
12		counter++;
08000280:	adds	r4, #1
13		counter++;
08000282:	adds	r4, #1
14		counter++;
08000284:	adds	r4, #1
15		counter++;
08000286:	adds	r4, #1

Question 1.6: Now have a look at PC (R15), do you notice by how much it increases?